Service – Inviare e ricevere messaggi tra nodi

Fondamenti di Robotica

Docenti: Loredana Zollo
Clemente Lauretti
Tutor: **Rita Molle**
email: rita.molle@unicampus.it

CAMPUS BIO-MEDICO UNIVERSITY OF ROME
Via Álvaro del Portillo, 21 - 00128 Rome - Italy
www.unicampus.it

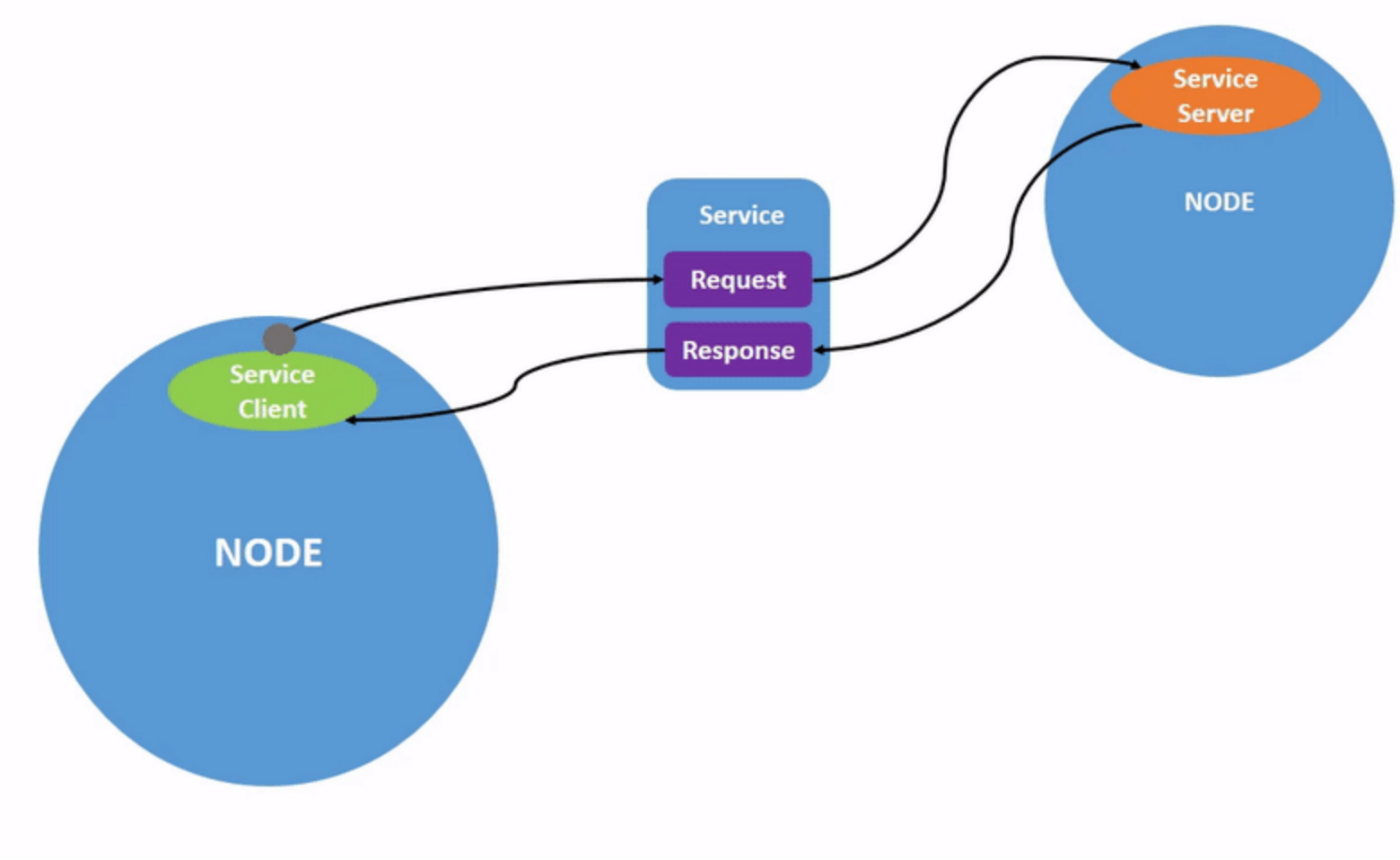
CREO LAB

UNIVERSITA' CAMPUS BIO-MEDICO DI ROMA

Outline

- Recap
- Esercitazione
- Scrivere un service server
- Scrivere un service client

Recap


Service
Server
NODE
Service
Request
Response
Service
Client
NODE

Recap

- Un service permette la comunicazione tra nodi tramite client/server
- Un service è definito da **name** e **interface** (request + response)
- Il name deve iniziare con una lettera
- server e client devono avere la stessa **interface**

Esempio di struttura di un file .service

```
# Request
Int64 a
int64 b
---
# Response
int64 sum
```

- Si usa il topic per pubblicare in maniera unidirezionale; si usano i services per avere una comunicazione client/server *(esempio: topic per inviare dati letti da un sensore; service per eseguire delle azioni quando si hanno richieste → start e stop di un robot)*

- Un solo server può avere più clients
- I clients non conoscono gli altri clients e il server → la comunicazione avviene tramite il service name con la corretta interface
- Un nodo può contenere più server e clients, ognuno con un service name diverso

E. Renard, *ROS 2 from Scratch: Get Started with ROS 2 and Create Robotics Applications with Python and C++*. Packt Publishing, Limited, 2024. [Online]. Disponibile su: https://books.google.it/books?id=5AsuEQAAQBAJ

Esercitazione:

- Scrivere un nodo service server che crea il service /play_motion_service
- Scrivere un nodo service client che utilizza il service /play_motion2
  - Lista tutte le motion disponibili
  - Verifica se una motion è pronta
  - Esegue motion tramite ros2 run (da codice)

Terminal:

```
ros2 service list
ros2 service type/play_motion2
ros2 interface show play_motion2_msgs/srv/ListMotions
```


E. Renard, *ROS 2 from Scratch: Get Started with ROS 2 and Create Robotics Applications with Python and C++*. Packt Publishing, Limited, 2024. [Online]. Disponibile su: https://books.google.it/books?id=5AsuEQAAQBAJ

Scrivere un service server – Python

Usare la funzione

create_service(*service_interface*, "*service_name*", self.callback_*service_name)*

Creare una callback

Dati dal Client — Vuota: da riempire

```
def callback_service_name(self, request: service_interface.Request, response:
service_interface.Response):
        self.counter_ = request.service_name
        self.get_logger().info("Reset counter to" + str(self.counter_))
        response.success = True
        response.message = "Success"
        return response
        msg.data = self.data
        self.publisher.publish(msg)
```

Scrivere un service client – Python

Per avere la comucazione service è necessario avere sia il server che il client
→ Nuovo nodo con il service client → invia la richiesta al service server e ottiene la risposta

Creare un file move_head_service.py

Che tipo di interface??? Si ragiona su richiesta e riposte

Importare il tipo di messaggio → interface

```
from play_motion2_msgs.srv import …
```

Nome pacchetto — **Nome della cartella** — **Classe dell'interface**

Creare il service client con la funzione

create_client(*service_interface*, "*service_name*")

Scrivere un service client – Python

Si invia la richiesta con call_async
Il server riceve e processa la richiesta
Il server restituisce una risposta al nodo del client
Il client riceve la risposta e la processa nella callback

E. Renard, *ROS 2 from Scratch: Get Started with ROS 2 and Create Robotics Applications with Python and C++*. Packt Publishing, Limited, 2024. [Online]. Disponibile su: https://books.google.it/books?id=5AsuEQAAQBAJ

Scrivere un service client – Python

Fare la call per il nodo

```
def callback_service_name(self, value):
    while not self.client_.wait_for_service(1.0):
        self.get_logger().warn("Waiting for service")
    request = service_interface.Request()
    request.reset_value = value
    future = self.client_.call_async(request)
    future.add_done_callback(self.callback_service_name_response)
```

Restituisce True quando trova il service, oppure False dopo il timeout (1.0)

Oggetto richiesta

Invia la richiesta e restituisce un Future object

Processare la risposta dal service

```
def callback_service_name_response(self, future):
    response = future.result()
    self.get_logger().info("Success flag:" + str(response.success))
    self.get_logger().info("Message:" + str(response.message))
```

Risposta

E. Renard, *ROS 2 from Scratch: Get Started with ROS 2 and Create Robotics Applications with Python and C++*. Packt Publishing, Limited, 2024. [Online]. Disponibile su: https://books.google.it/books?id=5AsuEQAAQBAJ